S2S-03: Step 3 — Design: Target Tenant Architecture

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 3 of 9 | **Phase:** Plan | **Step:** Design | **Created:** March 2026 | **Last Updated:** 10/15/2026

Overview

With discovery complete (Step 1) and a migration strategy selected (Step 2), this step designs the target tenant architecture. Good design prevents the most painful migration failures — naming collisions, broken data pipelines, overprivileged access, and orphaned integrations. Every design decision made here becomes a deployment instruction in Step 4 (Prepare).

This notebook produces five deliverables: an IAM architecture, a Grail bucket and OpenPipeline layout, a cloud integration mapping, a configuration deployment order, and an entity ID remapping strategy.

S2S Migration Framework

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → **3. Design**	Understand what exists, choose approach, design target
Upgrade	4. Prepare → 5. Execute → 6. Integrate	Build target, move config, connect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Roll out agents, enable teams, tune and decommission

> **You are here: Step 3 — Design.** You have inventoried the source tenant (Step 1) and selected a migration strategy (Step 2). Now you design the target tenant architecture before building anything.

Table of Contents

1. IAM Architecture Design
2. Grail Bucket and Retention Design
3. OpenPipeline Layout
4. Cloud Integration Mapping
5. Configuration Deployment Order
6. Entity ID Remapping Strategy
7. Step Completion Checklist

Prerequisites

Requirement	Details
Step 1 Complete	Source tenant discovery finished — entity inventory, configuration export, audit log analysis
Step 2 Complete	Migration strategy selected (consolidation, relocation, split, hosting-cloud change, or workload cloud change)
Source Tenant Access	Admin access with `settings.read`, `ReadConfig` scopes
Target Tenant	Provisioned Dynatrace SaaS environment
OAuth Client	`account-idm-read`, `account-idm-write`, `iam-policies-management` scopes (for IAM design validation)

Requirement	Details
Stakeholder Alignment	IAM owners, platform team, and security team available for design review

> The Terraform resources used below (`dynatrace_iam_group` , `dynatrace_iam_policy` , and for SLOs `dynatrace_platform_slo` — the classic `dynatrace_slo_v2` only where a tenant is still on classic SLOs) are documented with full worked examples in **AUTOM-04**'s consolidated resource catalog — this notebook shows the migration-specific design pattern, not the general resource shape.

1. IAM Architecture Design

IAM is the most politically complex part of a SaaS-to-SaaS migration. Groups, policies, and bindings must be rebuilt in the target. They are account-level objects, managed with Terraform (`dynatrace_iam_*`) or Monaco's dedicated `monaco account` commands — this series uses Terraform, for its state and drift detection.

> **Sources:** Monaco account configuration (DT docs) — *"Using Monaco, you can define users, service users, groups, policies, and boundaries as dedicated types in YAML configuration files."*; *"Account management requires OAuth credentials."*

Decision: Lift-and-Shift vs Redesign

Approach	When to Use	Pros	Cons
Lift-and-Shift	Simple relocation, no org changes	Fast, minimal disruption	Carries forward existing IAM debt
Redesign	Consolidation, M&A, compliance changes	Clean architecture, right-sized access	More planning, requires stakeholder alignment
Hybrid	Migrate critical access first, redesign later	Quick initial migration	Requires follow-up project

> **Recommendation:** For **consolidation**, always redesign. You are merging multiple IAM structures — lifting and shifting all of them creates overlapping, conflicting policies. Use the migration as the forcing function to rationalize group sprawl.
>
> For **relocation**, lift-and-shift is acceptable. The org structure is not changing, so the existing IAM model is still valid.

IAM Components

Component	Description	Scope	Tool
Groups	Collections of users with shared access needs	Account-level	Terraform or `monaco account`
Policies	Permission statements with `ALLOW` / `DENY` and `WHERE` clauses	Account, environment, or management zone	Terraform or `monaco account`
Bindings	Link policies to groups at a specific scope	Account, environment, or management zone	Terraform or `monaco account`

IAM Design Template

Use this template to map source IAM to target IAM:

Source Group(s)	Target Group	Policy Scope	Key Permissions
`us-east-admins` + `eu-west-admins`	`platform-admins`	Account	Full admin
`us-east-sre` + `eu-west-sre`	`sre-team`	Environment	Settings read/write, logs read

Source Group(s)	Target Group	Policy Scope	Key Permissions
`us-east-devs` + `eu-west-devs`	`app-developers`	Management zone	Logs read, metrics read, dashboards
`us-east-readonly` + `eu-west-readonly`	`observers`	Environment	Read-only across all data

Policy Design Pattern

```
resource "dynatrace_iam_policy" "sre_settings" {
  name            = "SRE Team - Settings Access"
  environment     = var.dynatrace_environment_id
  statement_query = &lt;&lt;-EOT
    ALLOW settings:objects:read, settings:objects:write
      WHERE settings:schemaId startsWith "builtin:alerting";
    ALLOW settings:objects:read, settings:objects:write
      WHERE settings:schemaId startsWith "builtin:anomaly-detection";
    ALLOW storage:logs:read, storage:spans:read, storage:metrics:read;
    ALLOW storage:buckets:read WHERE storage:table-name IN ("logs", "spans", "metrics");
  EOT
}
```

> **Warning:** IAM policies use `WHERE` clauses that reference schemas, buckets, and security contexts (e.g., `WHERE storage:bucket-name = "app_logs"`). Every table read also needs `storage:buckets:read`, which the IAM policy reference (DT docs) describes as *"Required additionally to a table permission"* — without it the table permissions read nothing. Design buckets and segments **before** writing policies — the `WHERE` clauses must reference resources that will exist in the target tenant.

Audit Source IAM Structure

Before designing target IAM, understand the current access model. Groups, policies, boundaries and bindings are **account-level** objects: *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings"*, which administrators view in Account Management > Settings > Audit log (Account Management audit logs (DT docs)). The same page says *"The Account Management audit log doesn't yet cover every account-level change. Some IAM configuration and login events aren't recorded."* They are not in the source environment's audit events. What the environment can tell you is who actually uses it:

```
// Who uses the source environment: successful sign-ins per user (last 30 days)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in `logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing, or matched an
// unrelated file-based audit log (a database .aud file on the validation tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind == "AUDIT_EVENT".
// Group, policy, boundary and binding changes are ACCOUNT-level. They are not in an
// environment's audit events; review them in Account Management &gt; Settings &gt; Audit log.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN" and event.outcome == "success"
| summarize {logins = count(), last_login = takeMax(timestamp)}, by:{user.id}
| sort logins desc
| limit 20
```

2. Grail Bucket and Retention Design

Grail buckets control where data is stored and how long it is retained. For consolidation scenarios, bucket naming is critical — multiple source tenants may have identically named buckets.

Bucket Naming Strategy

Scenario	Naming Pattern	Example
Consolidation	Prefix by source tenant or business unit	`us-east_app_logs`, `eu-west_app_logs`

Scenario	Naming Pattern	Example
Relocation	Keep original names (no collision risk)	`app_logs`, `infra_metrics`
Workload cloud change	Prefix by cloud provider or region (keeps the dual-cloud window separable)	`aws_app_logs`, `azure_app_logs`
Unified (consolidation)	Merge into single bucket with enrichment tags	`app_logs` with `source_tenant` tag

> **Design Decision:** Prefixed buckets are safer for initial migration. You can consolidate into unified buckets later once data flows are validated. Starting with separate buckets preserves the ability to roll back.

Bucket Design Template

Source Tenant	Source Bucket	Target Bucket	Retention	Notes
US-East	`default_logs`	`default_logs`	35 days	Default bucket — keep as-is
US-East	`app_logs`	`us-east_app_logs`	90 days	Custom bucket — prefix to avoid collision
EU-West	`app_logs`	`eu-west_app_logs`	360 days	PCI requirement — longer retention
US-East	`security_logs`	`security_logs`	365 days	Merge — same compliance requirement

Retention Policy Alignment

Environment	Typical Retention	Compliance Driver
Production	90–365 days	PCI-DSS, SOX, HIPAA
Staging	35 days	Cost optimization
Development	7–14 days	Cost optimization
Security/Audit	365+ days	Regulatory requirement

> **Compliance Note:** For regulated industries, verify that retention policies in the target tenant meet all compliance requirements **before** cutover. Reducing retention below the source tenant's policy may violate audit requirements.

Inventory Source Buckets

Query the source tenant to catalog all custom buckets and their retention settings:

```
// List all Grail buckets with retention and estimated size
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days, estimated_uncompressed_bytes
| fieldsAdd size_gb = estimated_uncompressed_bytes / 1073741824.0
| sort size_gb desc
```

3. OpenPipeline Layout

OpenPipeline processing rules control how data is enriched, extracted, routed, and masked. These rules reference Grail buckets as routing targets, so bucket design must be finalized before designing the OpenPipeline layout.

Processing Rule Types

Rule Type	What It Does	Migration Notes
Enrichment	Add fields from lookup tables or entity properties	Port as-is if field names match

Rule Type	What It Does	Migration Notes
Extraction	Parse structured data from unstructured content	Port as-is
Routing	Direct data to specific Grail buckets	**Update bucket references** to target names
Masking	Redact sensitive data (PII, credentials)	Port as-is — verify compliance requirements match

OpenPipeline Design Template

Source Pipeline	Rule Count	Target Pipeline	Bucket Target	Changes Required
`logs.default`	12	`logs.default`	`default_logs`	Update bucket references
`logs.security`	8	`logs.security`	`security_logs`	Port as-is (bucket name unchanged)
`spans.default`	5	`spans.default`	`default_spans`	Port as-is
`events.custom`	3	`events.custom`	`us-east_events`	Rename bucket reference

Coordinated Deployment Order

OpenPipeline and related resources must be deployed in strict dependency order:

Order	Resource	Why This Order
1	Grail buckets	Routing rules reference buckets — they must exist first
2	Enrichment rules (K8s labels → Grail tags)	Tags feed OpenPipeline conditions and downstream queries
3	OpenPipeline processing rules	Now routing targets and enrichment sources exist
4	Segments	Segment definitions may reference buckets and tags

> **Monaco note:** `monaco deploy` resolves these dependencies automatically within a single run. The order above is for teams using Terraform or manual deployment.

Audit Source OpenPipeline Rules

Query the source tenant's audit log to understand which OpenPipeline configurations are actively managed:

```
// Audit log: OpenPipeline configuration changes (last 30 days)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in `logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing, or matched an
// unrelated file-based audit log (a database .aud file on the validation tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind == "AUDIT_EVENT".
// OpenPipeline configuration is stored as builtin:openpipeline.* settings schemas.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter startsWith(details.dt.settings.schema_id, "builtin:openpipeline.")
| filter in(event.type, {"CREATE", "UPDATE", "DELETE"})
| summarize changes = count(), by:{details.dt.settings.schema_id, user.id}
| sort changes desc
| limit 10
```

4. Cloud Integration Mapping

Cloud connections are environment-scoped — credentials, IAM roles and monitoring scopes are recreated in the target, never moved. Two generations exist, and the design should pick one per provider explicitly:

- **Latest Dynatrace — Cloud Platform Monitoring** (AWS and Azure connections, Clouds app). Polling runs inside Dynatrace SaaS: *"no need to deploy ActiveGate compute resources for metric polling"*.

- **Dynatrace Classic integrations** (classic AWS/Azure integrations, the classic Azure log forwarder). Use these only as the fallback where the target cannot use the Latest connection yet.

> **Monaco download limitation:** Cloud provider credentials **cannot be exported** via `monaco download`. Monaco can deploy credential configurations but not extract existing ones. You must document and recreate credentials manually.

Integration Inventory Template

Document all active cloud integrations from the source tenant, and the target action for each:

Provider	Integration	Scope	Credential	Target action (Latest first)
AWS	Metrics + topology	Accounts, regions	Role created by the connection	Create an AWS connection **from the target**; it is created through CloudFormation
AWS	Logs	CloudWatch log groups	Firehose stream	Subscribe log groups to the Firehose streams the target's connection generates
Azure	Metrics + topology	Subscriptions	Service principal	Create an Azure connection **from the target** with a **new, dedicated** service principal
Azure	Logs / events	Resource logs, activity logs / resource events	Event Hubs / Event Grid	SaaS-based ingest via Event Hubs (logs) and Event Grid (events)
GCP	Cloud Monitoring	Projects	Service account	New key or service account for the target (not re-verified in this update — see the CLOUD series)

Credential Recreation Checklist

Provider	Credential	Source value	Target action	Owner
AWS	Connection stack	Document account IDs and regions	Create the target's AWS connection; deploy the CloudFormation template it produces in each account	Cloud team
AWS (classic fallback)	IAM role + external ID	Document role ARN	New role, or updated trust policy, carrying the **target's** external ID	Cloud team
Azure	Service principal	Document subscriptions and the principal in use	Create a **new** principal for the target — the docs say not to share one across environments, even inside the same Entra ID tenant	Cloud team
GCP	Service account key	Document project and SA	Generate a new key for the target	Cloud team
K8s	ActiveGate / DynaKube tokens	Not exportable	Generate new tokens in the target	Platform team

> **Sources:**
> - AWS Cloud Platform Monitoring (DT docs) — *"All AWS connection creation methods are powered by CloudFormation"*; *"Subscribe CloudWatch log groups to auto-generated Firehose streams for immediate ingestion and analysis."*
> - Azure Cloud Platform Monitoring (DT docs) — *"no need to deploy ActiveGate compute resources for metric polling within your Azure environment."*
> - Create your first Azure connection (DT docs) — *"Use a dedicated service principal exclusively for Dynatrace. Do not share it across Dynatrace environments or use it for other non-Dynatrace workloads."*

Workload Cloud Change Scenarios

When the workloads themselves change cloud, the monitoring stack changes with them, not only the credentials. For the AWS ↔ Azure case on Latest Dynatrace:

Concern	AWS	Azure
Connection	AWS connection (CloudFormation)	Azure connection (dedicated service principal)
Metrics namespace	`cloud.aws.*` (e.g. `cloud.aws.ec2.CPUUtilization.By.InstanceId`)	`cloud.azure.*` (e.g. `cloud.azure.microsoft_compute.virtualmachines.PercentageCPU`)
Log ingest	Firehose	Event Hubs
Container platform	EKS	AKS
Hosts	`cloud.provider == "aws"`	`cloud.provider == "azure"`

The metric keys above were read with the `metrics` command on the validation tenant (09/28/2026). For the full "retire AWS, move to Azure" sequence, see the appendix LAB **S2S-94**. GCP scenarios follow the same shape but were not re-verified in this update.

5. Configuration Deployment Order

This is the single most important reference for Step 4 (Prepare) and Step 5 (Execute). Configuration must be deployed in dependency order — resources that other resources reference must exist first. IAM is deployed **last** because policies use `WHERE` clauses that reference resources.

> **Monaco note:** `monaco deploy` resolves dependencies automatically. The order below is the reference model for understanding dependencies and for teams using Terraform.

Phase 1: Data Foundation — Deploy First

Order	Category	Dependencies	Monaco Type	Terraform Resource
1	Grail buckets	None — routing needs these first	`bucket`	`dynatrace_platform_bucket`
2	Enrichment rules (K8s labels → tags)	None — tags feed everything downstream	`settings`	`dynatrace_kubernetes_enrichment`
3	OpenPipeline processing rules	Buckets must exist as routing targets	`openpipeline`	`dynatrace_openpipeline_v2_*` (e.g. `dynatrace_openpipeline_v2_logs_pipelines`)
4	Segments	None	`segment`	`dynatrace_segment`

Phase 2: Gen2 (Classic) Configuration

Order	Category	Dependencies	Monaco Type
5	Management zones	None	`api` / `settings`
6	Auto-tags	None	`api` / `settings`
7	Request attributes	None	`api` / `settings`
8	Calculated service metrics	None	`api`
9	Custom services and detection rules	None	`api` / `settings`
10	Application detection rules	None	`api` / `settings`
11	Conditional naming rules	None	`api`
12	Anomaly detection settings	None	`settings`
13	Credential vault entries	None	`api`

Phase 3: Monitoring and Alerting

Order	Category	Dependencies	Monaco Type
14	Alerting profiles	None	`api` / `settings`
15	Notification rules + integrations	Alerting profiles	`api` / `settings`
16	Maintenance windows	Management zones (for scope)	`api` / `settings`
17	SLO definitions (modern SLO app; `slo-v2` is the Grail-based type, Monaco v2.22+)	Tags, segments (management zones only for classic SLOs)	`slo-v2`
18	Synthetic monitors	Notification rules, credential vault	`api`

Phase 4: User-Facing Configuration

Order	Category	Dependencies	Monaco Type
19	Classic dashboards	Management zones, auto-tags	`api`
20	Gen3 dashboards and notebooks	SLOs, tags, segments	`document`
21	Workflows and automations	Notification rules, SLOs	`automation`

Phase 5: Governance — Deploy Last

Order	Category	Dependencies	Tool
22	IAM policies	All resources above exist	Terraform or `monaco account`
23	IAM groups	Policies defined	Terraform or `monaco account`
24	IAM policy bindings	Groups + policies	Terraform or `monaco account`

> **Why IAM last?** IAM policies use `WHERE` clauses that reference schemas, buckets, and security contexts (e.g., `WHERE storage:bucket-name = "app_logs"`). If those resources do not exist yet, you cannot validate that the policies are correct. Deploy the resources first, then lock them down with IAM.

Validate Deployment Prerequisites

Before starting deployment, verify that the target tenant has the necessary foundation:

```
// Target tenant: verify buckets exist before deploying OpenPipeline rules
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days
| sort display_name asc
```

```
// Target tenant: verify enrichment rules are populating tags
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize tagged_records = count()
| fieldsAdd status = if(tagged_records &gt; 0, then: "Enrichment rules active", else: "No tagged records — check
enrichment config")
```

6. Entity ID Remapping Strategy

Entity IDs (e.g., `HOST-1A2B3C4D`, `SERVICE-5E6F7A8B`) are **not portable** between tenants. They are auto-generated and unique per tenant. Any configuration that references entity IDs must be updated before or after import.

Where Entity IDs Appear

Configuration	Example Reference	Impact If Not Remapped
Dashboard tile filters	`entityId("HOST-1A2B3C4D")`	Dashboard shows no data
SLO metric expressions	`type(SERVICE),entityId(SERVICE-5E6F7A8B)`	SLO evaluates to 0%
Notification rule filters	Entity-specific tag filters	Alerts never fire
Management zone rules	Entity-specific rules	Zone contains no entities
Workflow triggers	Entity ID in detected problem filter	Workflow never triggers

Remapping Strategy: Replace with Tags

The best strategy is to eliminate hardcoded entity IDs **before** migration by replacing them with tag-based selectors:

```
# BEFORE (entity ID — breaks on migration)
filter = "type(SERVICE),entityId(SERVICE-5E6F7A8B)"

# AFTER (tag-based — survives migration)
filter = "type(SERVICE),tag(app:checkout),tag(env:production)"
```

> **Recommendation:** Audit all dashboards and SLOs for hardcoded entity IDs in the source tenant **before** migration. Convert to tag-based filters. This makes configuration portable across tenants and resilient to future migrations.

Entity ID Lookup Pattern

For entity IDs that cannot be replaced with tags (rare), use entity name + type for lookup in the target tenant after agents are reporting:

```
// Look up a service by name in the target tenant to find its new entity ID (Smartscape)
smartscapeNodes "SERVICE", from:-7d
| filter contains(name, "checkout")
| fields name, id, id_classic
| limit 10

// id_classic carries the classic SERVICE-… identifier used by classic dashboards, SLOs and
// selectors. It prints the same value as id, but the two have different types: compare with
// toString(id) == id_classic, never id == id_classic (always false — see FAQ-25 §4).
// Classic fallback: fetch dt.entity.service | filter contains(entity.name, "checkout")
```

```
// Audit source tenant: Settings 2.0 objects written with hardcoded entity IDs (last 30 days)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in `logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing, or matched an
// unrelated file-based audit log (a database .aud file on the validation tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind == "AUDIT_EVENT".
// Scope: audit events carry the object payload (details.json_after) only for Settings 2.0
// objects, and only for objects changed inside the timeframe. Classic dashboards are not
// covered - search your configuration export for HOST-/SERVICE- IDs for a complete inventory.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter in(event.type, {"CREATE", "UPDATE"})
| filter contains(details.json_after, "HOST-") or contains(details.json_after, "SERVICE-")
| summarize references = count(), by:{details.dt.settings.schema_id, user.id}
| sort references desc
| limit 20
```

Entity ID Remapping Checklist

Item	Action	Priority
Dashboard tile filters	Replace `entityId()` with tag selectors	High — most visible to users
SLO metric expressions	Replace entity selectors with tag-based selectors	High — affects SLO evaluation
Management zone rules	Replace entity-specific rules with property/tag rules	Medium
Notification rule scope	Replace entity filters with tag filters	Medium
Workflow triggers	Update detected problem entity filters	Medium
Synthetic monitor entity scope	Update location and entity references	Low — locations are global

7. Step Completion Checklist

Before proceeding to Step 4 (Prepare), verify all design deliverables are complete:

Deliverable	Status	Owner	Notes
IAM architecture designed	☐	Security / Platform	Groups, policies, bindings documented
IAM approach selected	☐	Security / Platform	Lift-and-shift vs redesign decision made
Grail bucket layout designed	☐	Platform	Naming, retention, collision strategy documented
OpenPipeline layout designed	☐	Platform	Rule inventory, bucket references updated
Cloud integration mapping documented	☐	Cloud / Platform	All integrations inventoried, credential plan ready
Configuration deployment order planned	☐	Platform	24-step order reviewed and agreed
Entity ID remapping strategy defined	☐	Platform / App teams	Hardcoded IDs inventoried, tag replacement plan ready
Stakeholder sign-off	☐	All	Design review completed, no open blockers

> **Gate check:** Do not proceed to Step 4 until all deliverables are documented and reviewed. Design gaps discovered during execution are 10x more expensive to fix.

Next Step

Continue to **S2S-04: Step 4 — Prepare: Build the Target Tenant** to begin deploying the designed architecture into the target tenant.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~	**4. Prepare**	5. Execute → 6. Integrate → 7. Expand → 8. Enable → 9. Optimize

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.